

Plataforma de Chat Empresarial Distribuido Grupo 3

Integrantes: Andrés Guaminga, Bryan Merino, Juan Pindo

Curso: 3 B

Asignatura: Programacion Orientada a objetos

Fecha: 01 de noviembre de 2026

1. objetivo del sistema

Desarrollar una aplicación de Chat Empresarial con arquitectura cliente-servidor concurrente en el lenguaje de programación Go, diseñada para operar sobre la infraestructura de red corporativa distribuida que interconecta la Sede A y la Sede B de la organización. El sistema funcionará sobre una topología enrutada con OpenWrt, protocolo dinámico OSPF y segmentación por VLANs, alojando el servidor central en Go y el servicio de autenticación en la VLAN 20 de Servidores (10.10.20.0/24) de la Sede A, y atendiendo las conexiones de los clientes ubicados en la VLAN 10 de Usuarios (10.10.10.0/24 en Sede A y 10.10.110.0/24 en Sede B) y en la VLAN 99 de Administración.

La aplicación ofrecerá las siguientes operaciones cliente-servidor reales: (1) iniciar sesión validando credenciales contra el servicio de autenticación en FastAPI, (2) enviar mensajes de texto en tiempo real a una sala departamental, (3) recibir mensajes de forma asíncrona y concurrente sin bloquear la consola local, (4) consultar el listado de salas disponibles y usuarios activos, (5) crear nuevas salas de chat departamentales y (6) expulsar o desconectar administrativamente a usuarios del servidor. El sistema sirve directamente a los empleados de ambas sedes para su coordinación diaria sin depender de servicios de mensajería públicos externos que comprometan la confidencialidad corporativa, así como al personal de TI (Administradores) encargado de moderar el acceso y supervisar la disponibilidad del servicio.

2. Actores

De acuerdo con los lineamientos de modelado UML del proyecto, los integrantes del equipo de desarrollo no son actores del sistema. A continuación se definen los actores humanos que utilizan la aplicación y el servicio externo con el que se comunica:

- **Empleado (Usuario Estándar):** Personal operativo y administrativo de la Sede A y la Sede B que utiliza el cliente de chat en Go desde la red de usuarios (VLAN 10). Puede autenticarse con su usuario y contraseña, abrir una conexión TCP con el servidor, consultar la lista de salas activas y participantes conectados (/listar), unirse a una sala departamental, enviar mensajes de texto a sus compañeros y recibir en tiempo real los mensajes transmitidos dentro de su sala.
- **Administrador (Personal de TI):** Usuario con privilegios elevados dentro de la plataforma encargado de supervisar la operación del sistema. Hereda todas las capacidades de comunicación del actor Empleado y, adicionalmente, está autorizado para ejecutar operaciones administrativas protegidas: crear nuevas salas departamentales en el servidor (/crear-sala) y expulsar o cerrar la conexión de usuarios activos (/expulsar) ante mantenimientos o incumplimientos de políticas.

- **Servicio de Autenticación en FastAPI (Actor / Sistema Externo):** Servicio web independiente desplegado en la VLAN 20 con el cual se comunica el servidor de chat en Go mediante peticiones HTTP antes de autorizar cualquier sesión sobre el socket TCP. Puede recibir las credenciales enviadas durante el inicio de sesión, verificar el hash criptográfico (bcrypt) en su base de datos de usuarios, devolver el código HTTP 200 junto con el rol asignado (Empleado o Administrador) y el token de sesión, o rechazar los intentos inválidos devolviendo el código HTTP 401.

3. Alcance

Que realizara la aplicación

- **Comunicación cliente-servidor en equipos distintos:** Establecer conexiones mediante sockets TCP en el puerto 8080 entre el servidor central en Go y múltiples clientes ejecutados en hosts independientes a través de la red enrutada entre la Sede A y la Sede B.
- **Autenticación y validación de sesión activa:** Solicitar usuario y contraseña al iniciar el cliente, validar las credenciales contra el servicio en FastAPI y verificar que exista una sesión válida antes de procesar cualquier operación protegida.
- **Mensajería concurrente en tiempo real:** Enviar y recibir mensajes de texto simultáneamente entre múltiples usuarios conectados a una misma sala utilizando goroutines y sincronización segura en memoria.
- **Consulta de directorio en vivo:** Permitir a cualquier usuario autenticado listar las salas departamentales existentes y la cantidad de usuarios conectados en el servidor.
- **Administración basada en roles:** Permitir exclusivamente al rol Administrador la creación de nuevas salas de conversación y la expulsión de usuarios conectados.
- **Instrumentación para Estadística:** Registrar marcas de tiempo (timestamps) de envío/recepción, longitud en bytes de los mensajes y eventos de conexión para el análisis de Probabilidad y Estadística.

Que no hará la aplicación

- **Sin transmisión de voz o video:** La aplicación no procesará llamadas de audio VoIP ni videollamadas en tiempo real.
- **Sin transferencia de archivos multimedia pesados:** El sistema está enfocado en mensajería de texto empresarial (hasta 1024 bytes por trama) y no almacenará imágenes ni videos.

- **Sin gestión directa de base de datos de usuarios desde el cliente Go:** El registro inicial de empleados y el cambio de contraseñas se gestionan en el servicio de autenticación en FastAPI, no desde la consola del cliente Go.
- **Sin clientes móviles nativos:** No se desarrollarán aplicaciones para Android o iOS; el cliente se ejecutará en entorno de consola/escritorio multiplataforma compilado en Go.

4. Requisitos

En la siguiente tabla se especifican los requisitos funcionales (RF-SW-01, RF-SW-02 desglosado por cada operación del sistema, RF-SW-03 y RF-INT-01) y los requisitos no funcionales (RNF-01 a RNF-07), incluyendo para cada uno el actor correspondiente y un criterio de aceptación concreto y medible:

Código	Requisito	Actor	Criterio de aceptación
RF-SW-01	El cliente y el servidor de chat funcionan y se comunican en equipos distintos a través de la red.	Empleado / Administrador	Al ejecutar el servidor en un host de la VLAN 20 (10.10.20.x) y el cliente en otro equipo de la VLAN 10 (10.10.10.x o 10.10.110.x), la conexión TCP se establece en el puerto 8080 con 0 paquetes perdidos.
RF-SW-02.1	Iniciar sesión validando usuario y contraseña contra el servicio de autenticación.	Empleado / Administrador / Servicio FastAPI	Un usuario con credenciales válidas recibe HTTP 200 y token de sesión para acceder al chat; si las credenciales son inválidas, FastAPI devuelve HTTP 401 y el servidor rechaza la conexión.
RF-SW-02.2	Enviar un mensaje de texto a una sala de chat activa.	Empleado / Administrador	El mensaje llega a la consola de todos los miembros conectados a esa sala; si la sala destino no existe o el mensaje está vacío, se muestra un error y no se guarda ni retransmite nada.
RF-SW-02.3	Recibir mensajes de otros usuarios en tiempo real sin bloquear la escritura local.	Empleado / Administrador	Cuando un usuario envía un mensaje a la sala, el remitente, la hora y el contenido aparecen en la pantalla de los demás miembros mientras estos continúan escribiendo por teclado.

RF-SW-02.4	Consultar el listado de salas disponibles y usuarios conectados en el servidor.	Empleado / Administrador	Al enviar el comando /listar, el servidor muestra en pantalla los nombres exactos de todas las salas activas y el número total de usuarios con sesión iniciada en ese momento.
RF-SW-02.5	Crear una nueva sala de chat departamental en el servidor.	Administrador	Si un Administrador ejecuta /crear-sala con un nombre nuevo, la sala se agrega al servidor y aparece al consultar /listar; si el nombre ya existe, se muestra un error de duplicado y no se sobrescribe.
RF-SW-02.6	Expulsar y desconectar a un usuario activo del servidor de chat.	Administrador	Si un Administrador ejecuta /expulsar sobre un usuario conectado, el servidor cierra el socket de ese usuario; si lo ejecuta un Empleado, el servidor bloquea la orden y muestra 'Acceso Denegado'.
RF-SW-03	Las operaciones protegidas del sistema exigen una sesión válida activa.	Empleado / Administrador / Servicio FastAPI	Si un cliente intenta enviar mensajes o comandos por el socket TCP sin haberse autenticado o con una sesión vencida, el servidor rechaza la operación y solicita iniciar sesión.
RF-INT-01	El cliente Go, el servidor Go, el servicio FastAPI y la red entre sedes funcionan integrados.	Empleado / Administrador / Servicio FastAPI	Un usuario en la Sede A se autentica en FastAPI, entra a una sala en el servidor Go e intercambia 10 mensajes bidireccionales con un usuario de la Sede B con 0% de pérdida de mensajes.
RNF-01	El tiempo de entrega de un mensaje dentro de una sala no debe superar los 200 milisegundos.	Empleado / Administrador	En una prueba de envío de 100 mensajes entre sedes, al menos 95 mensajes registran una diferencia entre timestamp de envío y recepción menor o igual a 200 ms en el log del servidor.
RNF-02	El servidor debe soportar al menos 50 conexiones de clientes simultáneas mediante goroutines.	Empleado / Administrador	Al conectar 50 clientes concurrentes enviando 1 mensaje cada 2 segundos durante 60 segundos, las 50 sesiones se mantienen activas

			sin caídas y el uso de CPU no supera el 80%.
RNF-03	La validación de credenciales con FastAPI debe responder en un máximo de 1000 milisegundos.	Servicio FastAPI	En 50 solicitudes consecutivas de inicio de sesión desde el servidor Go hacia FastAPI, el 100% de las respuestas HTTP se reciben en un tiempo menor o igual a 1000 ms.
RNF-04	El 100% de las contraseñas en el servicio de autenticación deben almacenarse con hash bcrypt.	Servicio FastAPI	Al inspeccionar los registros de la tabla de usuarios en la base de datos de FastAPI, el 100% de los campos de contraseña contienen cadenas con formato bcrypt y 0 en texto plano.
RNF-05	Las comunicaciones entre cliente y servidor deben cifrarse mediante protocolo TLS.	Empleado / Administrador	En una captura de 50 paquetes TCP en el puerto 8080 realizada con Wireshark, el 100% de la carga útil (payload) se visualiza cifrada sin mostrar texto legible.
RNF-06	El tamaño máximo permitido por cada mensaje enviado será de 1024 bytes.	Empleado / Administrador	Al enviar un mensaje de 1500 bytes, el servidor rechaza el paquete, devuelve un aviso de límite de 1024 bytes superado y continúa operando en el 100% de las pruebas.
RNF-07	El servidor debe detectar desconexiones abruptas y liberar el socket en un máximo de 5 segundos.	Empleado / Administrador	Al cerrar forzosamente la terminal de un cliente activo, el servidor elimina al usuario de la lista de conectados y cierra el socket en menos de 5 segundos.

5. Casos de uso

A continuación se presenta una ficha detallada por cada caso de uso del sistema, especificando los actores participantes, los requisitos cubiertos, el flujo principal en pasos secuenciales y el manejo de excepciones ante fallos:

Ficha CU-01: Iniciar sesión

Elemento de la Ficha	Descripción Técnica
Actor(es)	Empleado, Administrador, Servicio de Autenticación en FastAPI.
Requisitos que cubre	RF-SW-01, RF-SW-02.1, RF-SW-03, RF-INT-01, RNF-03, RNF-04.

Flujo principal (pasos)	1. El usuario inicia la aplicación cliente en Go e ingresa su usuario y contraseña. 2. El cliente abre una conexión TCP hacia el servidor en Go (puerto 8080) y transmite las credenciales. 3. El servidor en Go envía una petición HTTP al servicio de autenticación en FastAPI con las credenciales. 4. FastAPI verifica el hash bcrypt en su base de datos y responde con HTTP 200, el rol del usuario y un token de sesión. 5. El servidor en Go registra la sesión como válida, asigna al cliente a la sala general y confirma el ingreso.
Excepciones (fallos)	- Contraseña o usuario incorrecto: FastAPI devuelve HTTP 401; el servidor informa 'Error: Credenciales inválidas' y cierra el acceso. - Servicio FastAPI no responde: Si la petición supera los 1000 ms (RNF-03), el servidor cancela la espera y muestra 'Error: Servicio de autenticación no disponible'. - Servidor Go apagado o inalcanzable: El cliente detecta el error de conexión TCP, informa que el servidor no responde y finaliza.

Ficha CU-02: Enviar mensaje

Elemento de la Ficha	Descripción Técnica
Actor(es)	Empleado, Administrador.
Requisitos que cubre	RF-SW-02.2, RF-SW-03, RNF-01, RNF-05, RNF-06.
Flujo principal (pasos)	1. El usuario escribe un mensaje de texto en la consola del cliente y presiona Enter. 2. El cliente envía el mensaje al servidor a través del socket TCP cifrado con TLS. 3. El servidor comprueba que la sesión del remitente sea válida y que el texto no supere los 1024 bytes. 4. El servidor identifica la sala activa del remitente y retransmite el mensaje a todos los usuarios conectados en esa sala. 5. El servidor guarda la marca de tiempo de envío para el registro estadístico de latencia.
Excepciones (fallos)	- Sesión vencida o no iniciada: El servidor rechaza el envío, responde 'Error: Sesión expirada, vuelva a iniciar sesión' y desconecta el socket. - Sala o destinatario no existe / mensaje vacío: El servidor muestra 'Error: La sala no existe o el mensaje está vacío' y no guarda ni difunde nada. - Mensaje supera los 1024 bytes: El servidor descarta la trama y devuelve un aviso de límite de longitud excedido.

Ficha CU-03: Recibir mensajes

Elemento de la Ficha	Descripción Técnica
Actor(es)	Empleado, Administrador.
Requisitos que cubre	RF-SW-02.3, RF-SW-03, RNF-01, RNF-02.
Flujo principal (pasos)	1. El cliente mantiene una goroutine activa escuchando permanentemente el flujo de lectura del socket TCP. 2. El servidor transmite un mensaje nuevo originado en la sala donde se encuentra el usuario. 3. El cliente recibe los datos, formatea la hora, el nombre del remitente y el texto, y los muestra en pantalla sin bloquear el teclado.
Excepciones (fallos)	- Servidor desconectado o corte de red: La goroutine detecta el cierre del flujo (EOF), muestra 'Error: Se perdió la conexión con el servidor' y cierra la aplicación cliente de forma segura.

Ficha CU-04: Listar salas y usuarios

Elemento de la Ficha	
Actor(es)	Empleado, Administrador.
Requisitos que cubre	RF-SW-02.4, RF-SW-03.
Flujo principal (pasos)	1. El usuario ingresa el comando /listar en la consola del cliente. 2. El servidor verifica que el cliente cuente con una sesión activa válida. 3. El servidor recorre en memoria el mapa de salas registradas y contabiliza las conexiones activas. 4. El servidor envía al cliente la lista de salas disponibles y el número de usuarios conectados.
Excepciones (fallos)	- Sesión vencida: El servidor rechaza la consulta y devuelve 'Error: Sesión no válida'. - Servidor no responde: Si se agota el tiempo de lectura del socket, el cliente muestra un error de tiempo de espera agotado.

Ficha CU-05: Crear sala de chat

Elemento de la Ficha	Descripción Técnica
Actor(es)	Administrador.
Requisitos que cubre	RF-SW-02.5, RF-SW-03.
Flujo principal (pasos)	1. El Administrador ingresa el comando /crear-sala <nombre> en su consola cliente. 2. El servidor valida que la sesión esté activa y verifica que el rol del usuario sea 'Administrador'. 3. El servidor comprueba que no exista otra sala registrada con el mismo nombre. 4. El servidor crea la instancia de SalaChat, la agrega al mapa central y confirma la creación al Administrador.
Excepciones (fallos)	- Usuario sin permisos (Rol Empleado): El servidor bloquea la operación y devuelve 'Acceso Denegado: se requiere rol

	Administrador'. - Sala ya existe: El servidor devuelve 'Error: Ya existe una sala con ese nombre' y no sobrescribe la sala existente.

Ficha CU-06: Expulsar usuario

Elemento de la Ficha	Descripción Técnica
Actor(es)	Administrador.
Requisitos que cubre	RF-SW-02.6, RF-SW-03, RNF-07.
Flujo principal (pasos)	1. El Administrador ejecuta el comando /expulsar <usuario> desde su consola. 2. El servidor comprueba que la sesión sea válida y que el solicitante tenga el rol 'Administrador'. 3. El servidor busca al usuario indicado dentro del mapa de conexiones activas. 4. El servidor notifica la desconexión al usuario afectado, cierra su socket TCP, lo retira de la sala y confirma la expulsión al Administrador.
Excepciones (fallos)	- Permiso insuficiente (Rol Empleado): El servidor rechaza la acción y devuelve 'Acceso Denegado'. - Usuario objetivo no existe o ya se desconectó: El servidor muestra 'Error: El usuario especificado no se encuentra conectado' y no realiza cambios.

6. Matriz de trazabilidad

En la siguiente matriz se vincula cada requisito funcional y no funcional con el caso de uso correspondiente, la clase y el método del Diagrama de Clases UML que lo atenderán, y el procedimiento con el que se probará su cumplimiento. Para cubrir la autenticación y el control de sesiones se agregó la clase ClienteAuth al diagrama de clases y se subió la versión corregida a la carpeta docs del repositorio:

Requisito	Caso de Uso	Clase UML	Método	Forma en que se probará
RF-SW-01	CU-01 a CU-06	ServidorChat / ClienteChat	Iniciar() / Conectar()	Ejecutar servidor y cliente en máquinas distintas y comprobar conexión TCP en el puerto 8080.

RF-SW-02.1	CU-01 Iniciar sesión	ClienteAuth / ServidorChat	ValidarCredenciales() / AutenticarCliente()	Probar login con clave correcta (HTTP 200 y acceso) y clave incorrecta (HTTP 401 y rechazo).
RF-SW-02.2	CU-02 Enviar mensaje	ClienteChat / SalaChat	Enviar() / DifundirMensaje()	Enviar mensaje a una sala activa y verificar recepción en los miembros; probar con sala inexistente.
RF-SW-02.3	CU-03 Recibir mensajes	ClienteChat / UIApp	EscucharServidor() / MostrarMensaje()	Enviar mensajes desde un segundo cliente mientras el primero escribe por teclado sin bloqueo.
RF-SW-02.4	CU-04 Listar salas y usuarios	ServidorChat	ListarSalasYUsuarios()	Ejecutar /listar y contrastar el listado en pantalla con las salas y sesiones activas reales.
RF-SW-02.5	CU-05 Crear sala de chat	ServidorChat / SalaChat	CrearSala()	Ejecutar /crear-sala como Administrador (crea la sala) y como Empleado (devuelve 'Acceso Denegado').
RF-SW-02.6	CU-06 Expulsar usuario	ServidorChat / ConexionCliente	ExpulsarCliente() / Cerrar()	Ejecutar /expulsar como Administrador sobre un

				cliente activo y verificar el cierre de su socket.
RF-SW-03	CU-02 a CU-06	ConexionCliente	TieneSesionValida()	Intentar enviar tramas por TCP sin iniciar sesión y comprobar que el servidor bloquee la operación.
RF-INT-01	CU-01 a CU-06	ServidorChat, ClienteAuth, SalaChat, ClienteChat	Flujo integral de métodos	Prueba extremo a extremo entre Sede A, servicio FastAPI, servidor Go y Sede B por la red OSPF.
RNF-01	CU-02, CU-03	SalaChat / Mensaje	DifundirMensaje()	Medir en el log la diferencia de timestamps de 100 mensajes enviados (comprobar <= 200 ms).
RNF-02	CU-01, CU-02, CU-03	ServidorChat	manejarCliente()	Conectar 50 clientes concurrentes con un script de prueba durante 60 s y verificar CPU < 80%.
RNF-03	CU-01 Iniciar sesión	ClienteAuth	ValidarCredenciales()	Medir el tiempo de respuesta de 50 peticiones HTTP hacia FastAPI (comprobar <= 1000 ms).

RNF-04	CU-01 Iniciar sesión	ClienteAuth (FastAPI)	ValidarCredenciales()	Consultar la tabla de usuarios en la base de datos y comprobar que el 100% tenga hash bcrypt.
RNF-05	CU-02, CU-03	ServidorChat / ClienteChat	Iniciar() / Conectar()	Capturar 50 paquetes en el puerto 8080 con Wireshark y verificar que el payload esté cifrado con TLS.
RNF-06	CU-02 Enviar mensaje	ConexionCliente / Mensaje	LeerMensaje() / ValidarLongitud()	Enviar una trama de 1500 bytes y comprobar el rechazo automático al superar los 1024 bytes.
RNF-07	CU-03, CU-06	ServidorChat / ConexionCliente	EliminarCliente() / Cerrar()	Cerrar abruptamente la terminal de un cliente y cronometrar que el socket se libere en < 5 s.

7. Cambios respecto a la Fase 1

En la siguiente tabla se registran las observaciones y mejoras identificadas respecto a la entrega de la Fase 1 y las correcciones aplicadas en esta Fase 2:

Aspecto / Observación de Fase 1	Corrección aplicada en Fase 2
Diagrama de Clases sin clase para el servicio de autenticación externo.	Se agregó la clase ClienteAuth con el método ValidarCredenciales() y se añadieron los atributos Rol y tokenSesion en ConexionCliente, actualizando el diagrama en la carpeta /docs.
Operaciones del sistema agrupadas de forma general.	Se desglosó el requisito RF-SW-02 en seis filas independientes (RF-SW-02.1 a RF-SW-02.6), una por

	cada operación del Chat Empresarial, diferenciando los permisos de Empleado y Administrador.
Requisitos No Funcionales sin umbrales numéricos medibles.	Se definieron métricas verificables para RNF-01 a RNF-07 (200 ms de latencia, 50 goroutines concurrentes, 1000 ms en FastAPI, hash bcrypt al 100%, cifrado TLS, límite de 1024 bytes y desconexión en 5 s).
Trazabilidad entre requisitos, casos de uso y clases.	Se elaboró la matriz completa de trazabilidad verificando que cada requisito tenga asignado su caso de uso, su clase, su método en Go y su prueba de aceptación.
Actualización del Tablero Kanban y estructura en GitHub.	Se actualizaron las tarjetas de trabajo en el tablero de GitHub Projects y se incorporó el apartado 'Fase 2' en el archivo README.md apuntando al documento PDF en /docs.

Declaración de uso de IA

El presente documento declaramos el uso de la herramienta Google Gemini exclusivamente como recurso de soporte técnico auxiliar para: revisar la redacción técnica y la estructura de las tablas de criterios de aceptación y matriz de trazabilidad. El análisis del problema, la selección de operaciones del Chat Empresarial, la estructura del código en Go y la administración del repositorio son trabajo propio del equipo. prompt: revisa este documento y realiza las correcciones necesarias tanto de estructura como de ortografia y como debo acomodar las tablas realizadas

Repositorio: https://github.com/Noobmaster69-w/Proyecto-Integrador-